SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

QR-Based Visitor Management System

Version 2.0.0

Field	Details
Document Type	Software Requirements Specification (SRS) Documentation
System Name	QR-Based Visitor Management System
Version	v2.0.0
Prepared By	Amit Kushwaha
Department	MP Online Limited
Date	27 April 2026
Project Type	Internship Project

Document Revision History

Version	Date	Author	Description
1.0	February 2026	Intern	Initial SRS draft — core requirements
1.5	March 2026	Intern	Added multi-branch support, QR validity options
2.0	April 2026	Intern	Final — v2 features: photo capture, audit search, profile update

Table of Contents

#	Section	Page
1	Introduction	3
1.1	Purpose of Document	3
1.2	Project Scope	3
1.3	Definitions & Acronyms	3
1.4	References	3
2	Overall Description	4
2.1	Product Perspective	4
2.2	User Classes & Characteristics	4
2.3	Operating Environment	5
2.4	Design & Implementation Constraints	5
2.5	Assumptions & Dependencies	5
3	System Context — Context Diagram	6
4	Use Case Diagram	7
5	Functional Requirements	8
5.1	FR-AUTH — Authentication Module	8
5.2	FR-VIS — Visitor Registration Module	8
5.3	FR-SCAN — QR Scan Module	9
5.4	FR-ADMIN — Admin Module	9
5.5	FR-SUPER — Super Admin Module	10
6	Non-Functional Requirements	11
7	Activity Diagram	12
8	State Transition Diagram	13
9	Functional Modules Overview	14
10	External Interface Requirements	15
11	System Constraints & Limitations	16
12	Future Enhancements	16

1. Introduction

1.1 Purpose of Document

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements for the QR-Based Visitor Management System (VMS) v2.0, developed as an internship project. It serves as the reference agreement between the development team and stakeholders, documenting what the system does, who uses it, under what conditions, and what constraints apply.

This document is intended for: the development team, project supervisors, internship evaluators, and any future maintainers of the system.

1.2 Project Scope

The VMS is a full-stack web application that digitizes visitor entry and exit management for office premises. The system replaces manual paper-based visitor registers with a secure, QR-code-driven digital workflow. Key scope items:

- Visitor registration with photo capture and automatic QR code generation
- QR code delivery via email (SMTP) and on-screen display
- Guard-operated QR scanning for check-in and check-out
- Role-based dashboards for guards, admins, and super admins
- Multi-branch organizational support
- Comprehensive, immutable audit logging

Out of scope: mobile native app, visitor self-registration portal, SMS delivery, biometric authentication.

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
VMS	**Visitor Management System — the software described in this document**
QR Code	**Quick Response code — 2D barcode encoding the visit UUID token**
JWT	**JSON Web Token — stateless auth token issued on login (HS256 signed)**
RBAC	**Role-Based Access Control — permissions controlled by user role**
ORM	**Object-Relational Mapper — SQLAlchemy used to abstract DB queries**
SMTP	**Simple Mail Transfer Protocol — used to send QR codes via Gmail**
UUID	**Universally Unique Identifier — random token embedded in each QR**
SRS	**Software Requirements Specification — this document**
API	**Application Programming Interface — REST endpoints exposed by backend**
ASGI	**Async Server Gateway Interface — Uvicorn serves the FastAPI app**
bcrypt	**Password hashing algorithm used to store passwords securely**
DI	**Dependency Injection — FastAPI pattern used for auth and DB sessions**
Guard	**System role: security personnel who scan QR codes at entry/exit**
Admin	**System role: office admin who manages visitors and views reports**

Term	Definition
Super Admin	**System role: full access including users, branches, audit logs**

1.4 References

- FastAPI Documentation — https://fastapi.tiangolo.com
- React.js Documentation — https://react.dev
- SQLAlchemy 2.0 Docs — https://docs.sqlalchemy.org
- Pydantic v2 Documentation — https://docs.pydantic.dev
- python-jose (JWT) — https://github.com/mpdavis/python-jose
- html5-qrcode — https://github.com/mebjas/html5-qrcode
- IEEE Std 830-1998 — IEEE Recommended Practice for SRS

2. Overall Description

2.1 Product Perspective

VMS v2.0 is a standalone web application consisting of two independent tiers that communicate via REST API:

- Frontend: React.js Single Page Application (SPA) running in the browser at http://localhost:3000
- Backend: FastAPI (Python) application running via Uvicorn ASGI at http://localhost:8000
- Database: SQLite for development; PostgreSQL for production (switched via DATABASE_URL env var)

The system does not integrate with any third-party visitor management platforms. All data is stored within the organization's own database. The only external dependency is the Gmail SMTP server for email delivery.

2.2 User Classes & Characteristics

User Class	Role Value	Technical Level	Primary Functions	Access Level
Security Guard	**guard**	**Low — minimal training needed**	**Register visitors, scan QR at gate**	**Restricted**
Office Admin	**admin**	**Medium — computer literate**	**View dashboard, reports, visitor list**	**Elevated**
Super Admin	**super_admin**	**High — system administrator**	**Manage users, branches, audit logs**	**Full**
Visitor	**— (external)**	**None — receives email with QR**	**Present QR at entry gate**	**None (external)**
Email System	**— (external)**	**N/A — Gmail SMTP**	**Deliver QR emails to visitors**	**External**

2.3 Operating Environment

Component	Requirement
Backend OS	**Linux / Windows / macOS (any OS supporting Python 3.10+)**
Backend Runtime	**Python 3.10 or higher**
Backend Server	**Uvicorn ASGI (development), Gunicorn + Uvicorn (production)**
Frontend Browser	**Chrome 90+, Firefox 88+, Edge 90+ (requires camera access for QR scan)**
Database (Dev)	**SQLite 3.x — file-based, no setup required**
Database (Prod)	**PostgreSQL 14+ — via DATABASE_URL environment variable**
Email	**Gmail SMTP (smtp.gmail.com:587) — App Password required**
Network	**Local network or internet access; CORS allows localhost:3000 and :5173**
Node.js	**v18+ required for React frontend build tooling (npm)**

2.4 Design & Implementation Constraints

- The system uses JWT for authentication — tokens expire after 480 minutes (8 hours) by default
- QR codes contain UUID v4 tokens — non-sequential and cryptographically random
- QR expiry is configurable via QR_EXPIRY_HOURS in .env (default: 24 hours)
- All passwords are stored as bcrypt hashes — plain text passwords are never persisted

- The audit log table has no DELETE endpoint — records are immutable by design
- Photo uploads are stored as files on disk under uploads/photos/ — not in the database
- The frontend communicates only with the backend via REST API — no direct DB access
- CORS is restricted to localhost:3000 and localhost:5173 in development

2.5 Assumptions & Dependencies

- The organization has at least one branch created in the database before guards can register visitors
- The super admin seeds the system on first startup using FIRST_ADMIN_EMAIL and FIRST_ADMIN_PASSWORD in .env
- Email delivery assumes a valid Gmail App Password is configured in SMTP_USER and SMTP_PASSWORD
- The browser accessing the scanner page must have a physical or virtual camera device
- It is assumed that QR code validity periods configured by admins are appropriate for the organization's security policy
- Production deployment assumes Nginx as a reverse proxy in front of Uvicorn

3. System Context — Context Diagram (Level 0 DFD)

The context diagram below shows the VMS system as a single black box, surrounded by all external entities that interact with it. It defines the system boundary and the data flows crossing that boundary.

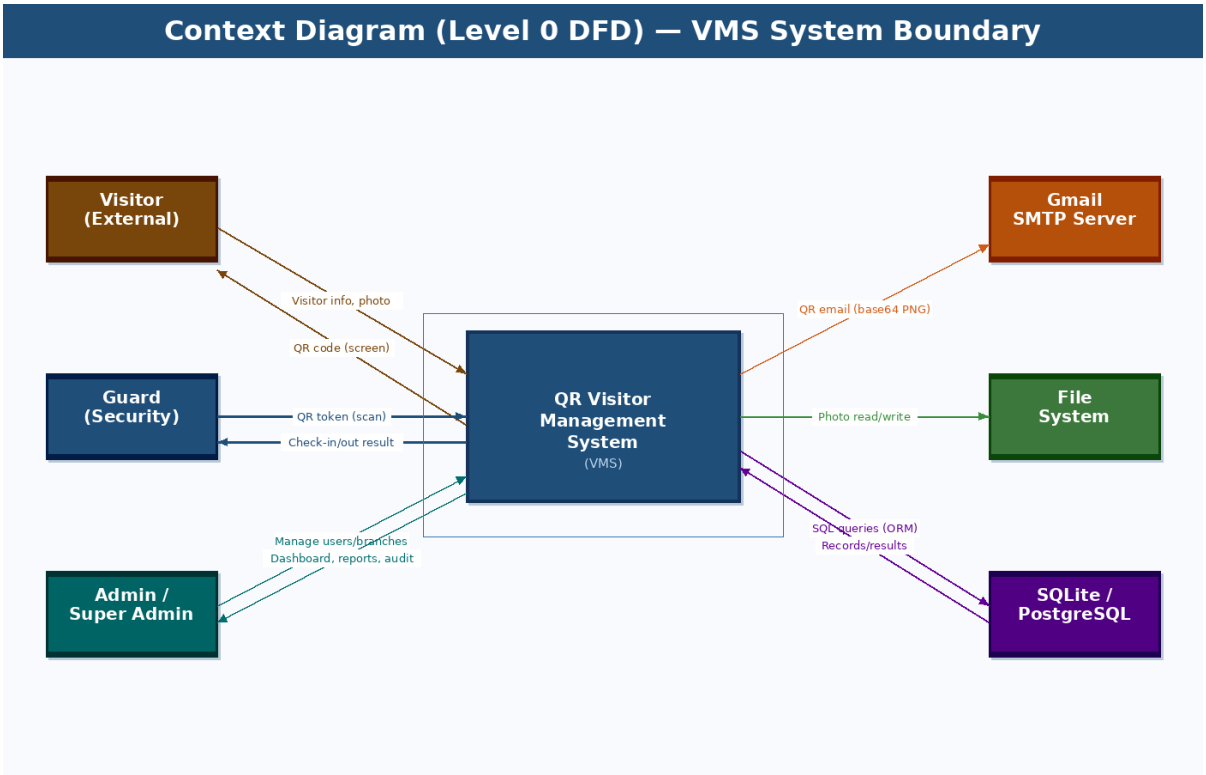

Figure 1: Context Diagram — VMS System Boundary & External Entities

3.1 External Entities

Entity	Type	Data Into System	Data From System
Visitor	**Human (external)**	**Visitor info, photo, visit purpose**	**QR code (on screen or email)**
Guard	**Human (internal user)**	**QR token (camera scan)**	**Check-in/out result, visitor name**
Admin	**Human (internal user)**	**Filter queries, date ranges**	**Dashboard stats, reports**
Super Admin	**Human (internal user)**	**User/branch create requests, audit queries**	**User lists, audit log records**
Gmail SMTP Server	**External service**	**(receives outbound email)**	**Email delivery status**
File System	**Infrastructure**	**Photo binary files (upload)**	**Photo files (read for display)**
SQLite/PostgreSQL	**Infrastructure**	**SQL INSERT/UPDATE queries**	**Query result sets**

4. Use Case Diagram

The use case diagram below shows all actors and the system functions they interact with. Use cases are grouped by actor role and color-coded for clarity.

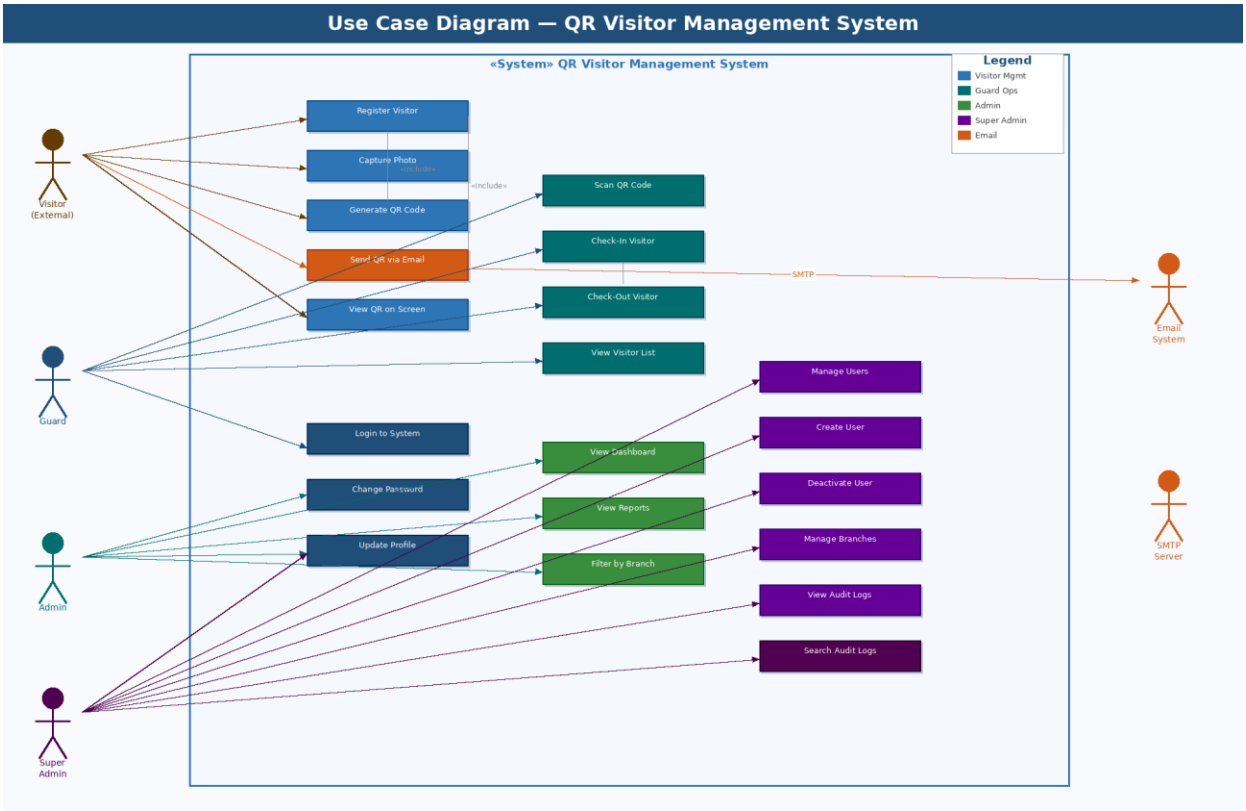

Figure 2: Use Case Diagram — All Actors & System Functions

4.1 Use Case Summary

Use Case	Actor(s)	Brief Description
Register Visitor	**Guard, Admin**	**Fill form with name, phone, email, purpose, host, photo**
Generate QR Code	**System (auto)**	**Create UUID token, render base64 QR PNG, set expiry**
Send QR via Email	**System (auto)**	**SMTP email with inline QR to visitor email**
Scan QR Code	**Guard**	**Camera decodes QR; system validates token and expiry**
Check-In Visitor	**Guard**	**First scan: status registered→checked_in, record timestamp**
Check-Out Visitor	**Guard**	**Second scan: status checked_in→checked_out, calc duration**
View Dashboard	**Admin, Super Admin**	**Real-time stats: inside now, today, this week, avg duration**
View Reports	**Admin, Super Admin**	**30-day visit trend, top hosts, branch breakdown**
Manage Users	**Super Admin**	**Create, list, deactivate system users (any role)**

Use Case	Actor(s)	Brief Description
Manage Branches	**Super Admin**	**Create and list office branch records**
View Audit Logs	**Super Admin**	**Paginated, searchable log of all system actions**
Login to System	**Guard, Admin, Super Admin**	**Email + password → JWT token returned**
Update Profile	**All users**	**Change own name, phone, department**
Change Password	**All users**	**Verify current password, set new bcrypt hash**

5. Functional Requirements

Each functional requirement is identified by a unique ID, priority (High/Medium/Low), and the API endpoint or UI component that implements it.

5.1 FR-AUTH — Authentication Module

ID	Requirement	Priority	Implementation
FR-AUTH-01	System shall authenticate users via email + password and return a JWT token	High	POST /api/auth/login
FR-AUTH-02	JWT token shall contain user_id, role, branch_id and expire after 480 minutes	High	security.py: create_access_token()
FR-AUTH-03	All protected routes shall reject requests without a valid Bearer token (401)	High	dependencies.py: get_current_user()
FR-AUTH-04	System shall enforce RBAC — unauthorized role access returns 403 Forbidden	High	dependencies.py: require_roles()
FR-AUTH-05	Users shall be able to view their own profile information	Medium	GET /api/auth/me
FR-AUTH-06	Users shall be able to update their name, phone, and department	Medium	PUT /api/auth/profile
FR-AUTH-07	Users shall be able to change their password after verifying the current one	Medium	POST /api/auth/change-password
FR-AUTH-08	Login events shall be recorded in audit_logs with user_id and IP address	Medium	auth.py: log_action()

5.2 FR-VIS — Visitor Registration Module

ID	Requirement	Priority	Implementation
FR-VIS-01	System shall allow registration of a visitor with name, phone, email, company, purpose, host, notes	High	POST /api/visitors/register
FR-VIS-02	System shall accept a base64 photo (webcam capture or file upload) and save it to disk	High	visit_service._save_photo()
FR-VIS-03	System shall match existing visitor by phone number to avoid duplicates; update name/email if changed	High	visit_service.register()
FR-VIS-04	System shall generate a UUID v4 token and render a QR PNG (base64 data URI)	High	qr.py: generate_token(), build_qr_image()
FR-VIS-05	System shall support configurable QR validity: hourly (custom hours), daily (24h), weekly (168h), monthly (720h)	High	schemas.py: QR_VALIDITY_HOURS
FR-VIS-06	If visitor email is provided and SMTP is enabled, system shall send QR code via email	High	email.py: send_qr_email()
FR-VIS-07	API response shall indicate whether email was sent (email_sent: true/false)	Medium	visit_service.register() return

ID	Requirement	Priority	Implementation
FR-VIS-08	System shall return the QR base64 image and expiry in the registration response for on-screen display	High	schemas.py: VisitOut
FR-VIS-09	Visitor registration shall be logged in audit_logs	Medium	visit_service: log_action()
FR-VIS-10	Guard/Admin shall be able to list all registered visitors with pagination	Medium	GET /api/visitors

5.3 FR-SCAN — QR Scan Module

ID	Requirement	Priority	Implementation
FR-SCAN-01	System shall provide a browser-based QR scanner using html5-qrcode library	High	ScannerPage.js
FR-SCAN-02	On first scan: if status=registered and QR not expired → set status=checked_in, record timestamp	High	visit_service.scan()
FR-SCAN-03	On second scan: if status=checked_in → set status=checked_out, calculate duration_mins	High	visit_service.scan()
FR-SCAN-04	System shall reject scans where QR token is not found (404)	High	visit_service.scan()
FR-SCAN-05	System shall reject scans where qr_expires < current UTC time (400 Bad Request)	High	qr.py: validate_qr()
FR-SCAN-06	Scan result shall return visitor name, phone, host, purpose, action, and timestamp	High	schemas.py: ScanResult
FR-SCAN-07	All scan events (check-in and check-out) shall be logged in audit_logs	High	visit_service: log_action()
FR-SCAN-08	scanned_by field shall record the guard's user_id for accountability	Medium	visit_service.scan()

5.4 FR-ADMIN — Admin Module

ID	Requirement	Priority	Implementation
FR-ADMIN-01	Admin shall view real-time dashboard: currently inside count, today total, week total, avg duration	High	GET /api/admin/dashboard
FR-ADMIN-02	Dashboard shall list all currently inside visitors with name, company, purpose, check-in time	High	visit_service.get_dashboard()
FR-ADMIN-03	Dashboard shall show purpose breakdown (meeting/delivery/interview etc.) for today	Medium	visit_service.get_dashboard()
FR-ADMIN-04	Dashboard shall show recent check-in/check-out activity (last 15 records)	Medium	visit_service.get_dashboard()

ID	Requirement	Priority	Implementation
FR-ADMIN-05	Admin shall access visit reports: daily counts for last N days (7–365), top hosts	Medium	GET /api/admin/report
FR-ADMIN-06	Dashboard and reports shall support optional branch_id filter for multi-branch orgs	Medium	query param: branch_id
FR-ADMIN-07	Admin shall be able to view list of all branches	Low	GET /api/admin/branches

5.5 FR-SUPER — Super Admin Module

ID	Requirement	Priority	Implementation
FR-SUPER-01	Super admin shall create new system users with any role and branch assignment	High	POST /api/admin/users
FR-SUPER-02	Super admin shall list all system users with pagination	High	GET /api/admin/users
FR-SUPER-03	Super admin shall deactivate (soft-delete) any user account	High	DELETE /api/admin/users/:id
FR-SUPER-04	Super admin shall create new office branches with name, city, address	High	POST /api/admin/branches
FR-SUPER-05	Super admin shall view paginated audit logs in reverse chronological order	High	GET /api/admin/audit-logs
FR-SUPER-06	Super admin shall search audit logs by user_id, action, entity, date, date range, IP address	High	GET /api/admin/audit-logs/search
FR-SUPER-07	Audit log records shall never be modifiable or deletable via any API	High	No DELETE/PUT endpoint exists
FR-SUPER-08	System shall auto-seed the first super admin on startup using .env credentials	High	init_db.py: seed_first_admin()

6. Non-Functional Requirements

ID	Category	Requirement	Metric / Verification
NFR-01	Performance	API response time for CRUD operations	< 300ms under normal load
NFR-02	Performance	QR scan endpoint response time	< 200ms (indexed qr_token lookup)
NFR-03	Security	Passwords must never be stored in plaintext	bcrypt hash verified in code review
NFR-04	Security	All protected routes require valid JWT	Unit tests: 401 on missing/expired token
NFR-05	Security	QR tokens must be non-guessable	UUID v4 (122 bits entropy)
NFR-06	Security	SQL injection must be prevented	SQLAlchemy ORM — parameterized queries only
NFR-07	Security	Input must be validated before processing	Pydantic schema validation on all endpoints
NFR-08	Reliability	System must handle SMTP failure gracefully	email_sent=false returned; no crash
NFR-09	Reliability	Database session must be closed after each request	get_db() dependency uses try/finally
NFR-10	Scalability	System must support multiple branches	branch_id FK on all relevant tables
NFR-11	Maintainability	Config must be externalized	All settings in .env via pydantic-settings
NFR-12	Maintainability	Code layered: routes → services → models	Verified in folder structure review
NFR-13	Usability	Guard UI must be operable without training	3-step form; large scan button
NFR-14	Usability	UI must support dark and light mode	ThemeContext.js with CSS variables
NFR-15	Auditability	Every state change must be traceable	audit_logs records user_id, action, IP
NFR-16	Portability	System must run on SQLite (dev) and PostgreSQL (prod)	DATABASE_URL env variable switches adapter

7. Activity Diagram

The activity diagram below shows two parallel swim lanes: (1) the visitor registration flow on the left, and (2) the QR scan check-in/check-out flow on the right. Decision diamonds show conditional branching.

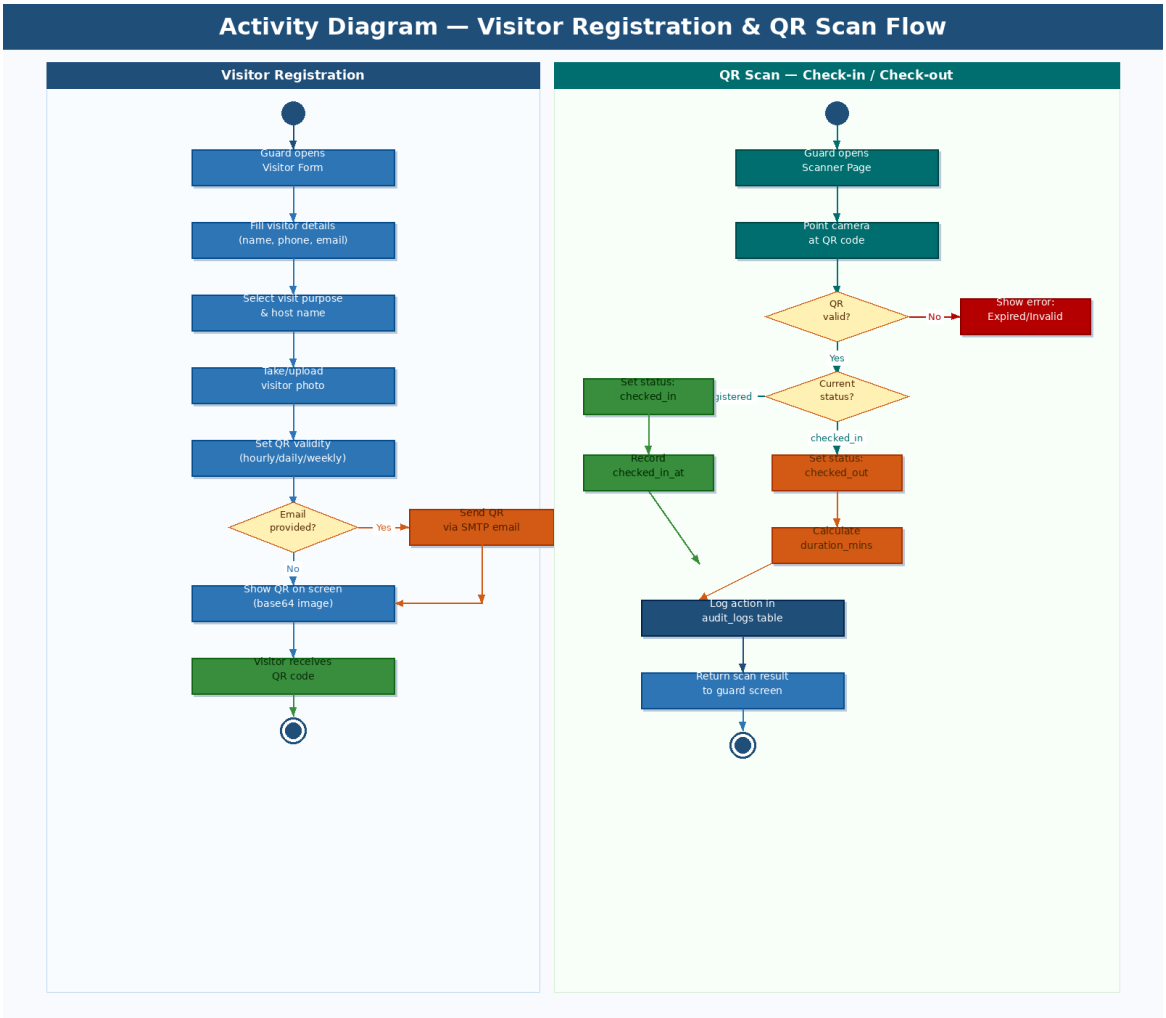

Figure 3: Activity Diagram — Registration Flow (left) & QR Scan Flow (right)

7.1 Registration Flow Steps

- Guard opens Visitor Form page on the React frontend
- Guard fills visitor details: name, phone, email (optional), company, purpose, host name, notes
- Guard captures photo via webcam or uploads a file
- Guard selects QR validity period (hourly/daily/weekly/monthly)
- System checks if email is provided — if yes, sends QR via SMTP email
- QR code is always displayed on screen for the guard to show the visitor

7.2 QR Scan Flow Steps

- Guard opens Scanner page — browser activates camera via html5-qrcode
- Guard points camera at visitor's QR code — decoded token sent to backend
- Backend validates token: checks qr_token exists and qr_expires > NOW()

- If status=registered: transition to checked_in, record checked_in_at timestamp
- If status=checked_in: transition to checked_out, calculate duration_mins
- All scan events logged to audit_logs with guard's user_id and timestamp

8. State Transition Diagram — Visit Status Lifecycle

Every visit record goes through a defined lifecycle tracked by the status column. The state transition diagram below shows all valid states, transitions, and the conditions that trigger each transition.

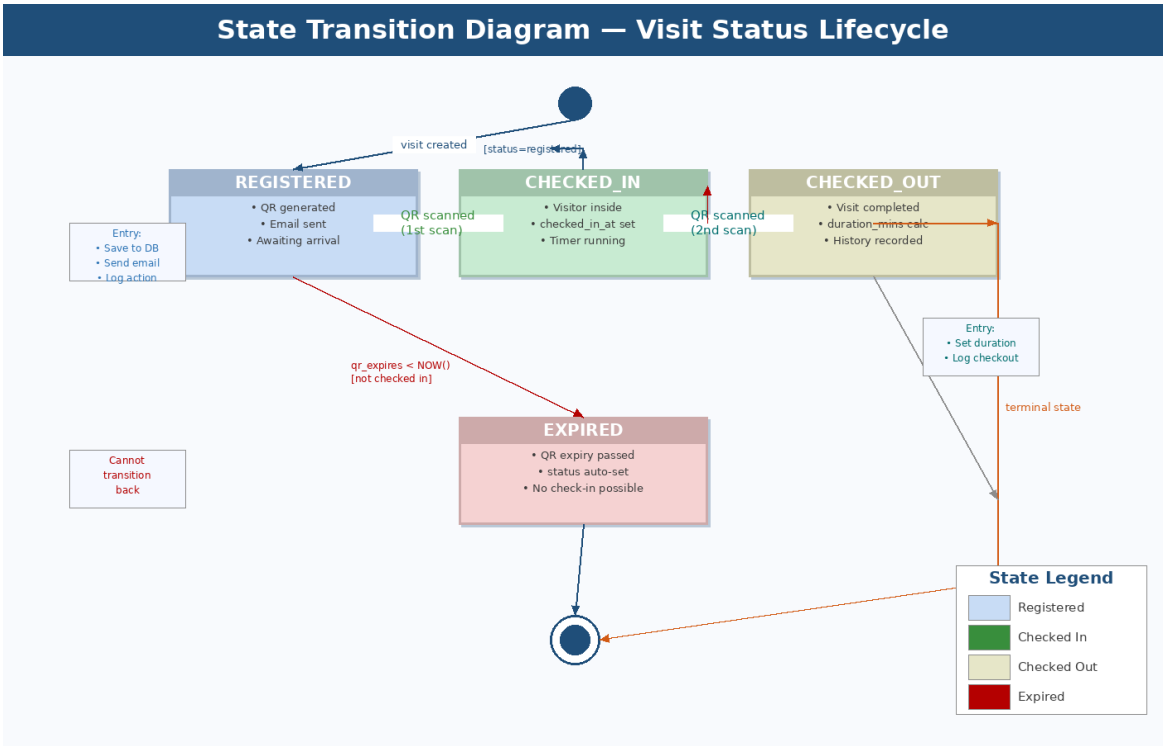

Figure 4: State Transition Diagram — Visit Status (registered → checked_in → checked_out / expired)

8.1 State Descriptions

State	Meaning	Entry Condition	Exit Condition(s)
registered	Visit created; QR generated; visitor not yet arrived	visit_service.register() completes successfully	QR scanned (→checked_in) OR qr_expires passes (→expired)
checked_in	Visitor scanned in; currently inside the building	First QR scan when status=registered and not expired	Second QR scan (→checked_out)
checked_out	Visit completed; visitor has left the building	Second QR scan when status=checked_in	Terminal — no further transitions possible
expired	QR expiry passed; visitor never checked in	qr_expires < NOW() at scan time	Terminal — no further transitions possible

8.2 Transition Rules

- Transitions are one-directional — no state can be reversed once set
- checked_out and expired are terminal states — no API can move a visit out of these states
- The scan endpoint (POST /api/visits/scan/:token) handles all valid transitions automatically
- duration_mins is calculated as integer minutes between checked_in_at and checked_out_at

9. Functional Modules Overview

The diagram below shows all five functional modules of the VMS backend, their exposed API endpoints, and the cross-cutting concerns that apply to all modules.

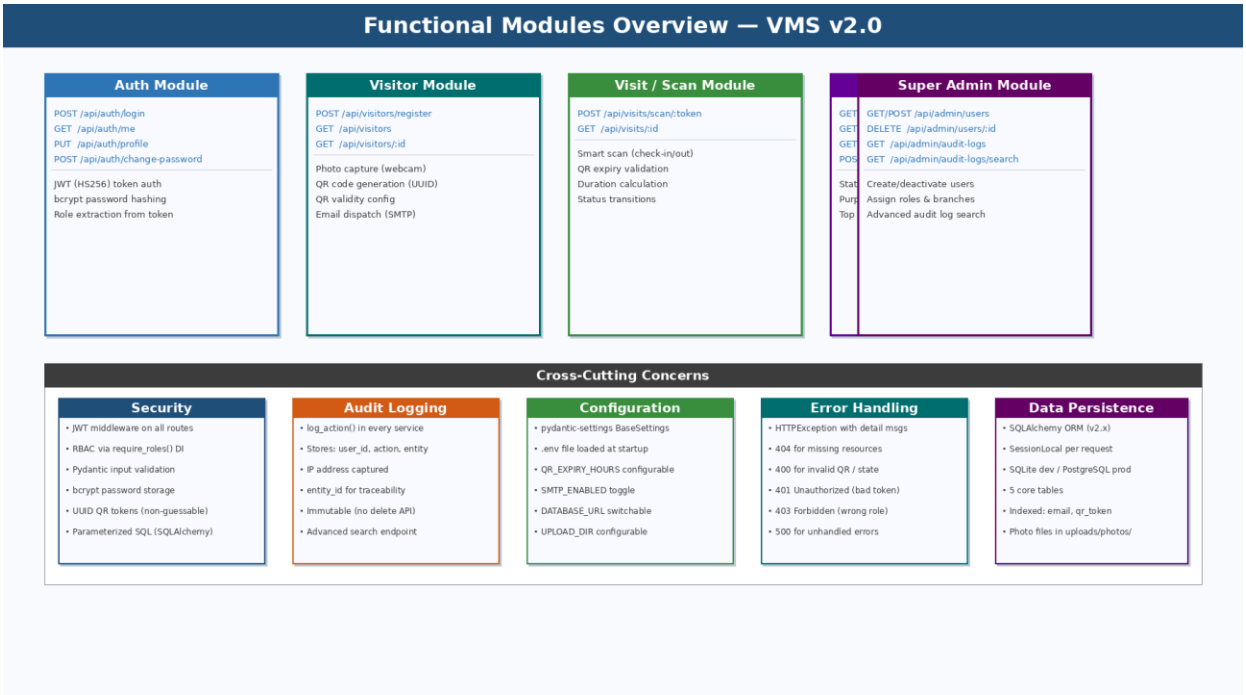

Figure 5: Functional Modules Overview — VMS v2.0

9.1 Module Responsibilities

Module	File(s)	Key Responsibility
Auth	**auth.py, security.py, dependencies.py**	**Login, JWT issue/verify, profile update, password change, RBAC enforcement**
Visitor	**visitor_routes.py, visit_service.py**	**Register visitor, save photo, generate QR, send email, list visitors**
Visit / Scan	**visit_routes.py, visit_service.py**	**Process QR scan, manage check-in/out state transitions, compute duration**
Admin	**admin_routes.py, visit_service.py**	**Dashboard stats, visit reports, branch listing**
Super Admin	**admin_routes.py, user_service.py**	**User CRUD, branch creation, audit log queries and advanced search**

10. External Interface Requirements

10.1 User Interface Requirements

- UI shall be a Single Page Application (SPA) built with React.js v18
- UI shall support dark mode and light mode toggled via ThemeContext
- Guard-facing pages (register, scan) shall have large, tap-friendly buttons suitable for touch screens
- QR code display shall render the base64 PNG image at sufficient size for camera scanning
- All forms shall show validation errors inline without full-page reload
- Loading states shall display a Loader spinner component during API calls

10.2 API Interface Requirements

Property	Specification
Base URL (dev)	**http://localhost:8000/api**
Protocol	**HTTP/1.1 REST; HTTPS required in production**
Auth Header	**Authorization: Bearer <JWT token>**
Request Format	**application/json (JSON body) or multipart/form-data (file upload)**
Response Format	**application/json — { data } on success, { detail } on error**
Interactive Docs	**Available at http://localhost:8000/docs (Swagger UI — FastAPI built-in)**
CORS Allowed Origins	**http://localhost:3000, http://localhost:5173 (dev)**
HTTP Status Codes	**200, 201, 400, 401, 403, 404, 410, 422, 500**

10.3 Email Interface Requirements

- Email delivery uses Gmail SMTP at smtp.gmail.com port 587 with STARTTLS
- Authentication requires a Gmail App Password (16-character token) — not the account password
- QR image is embedded inline in HTML email as a base64 data URI (no external image host required)
- Email includes: visitor name, host name, QR image, and QR expiry date/time
- If SMTP_ENABLED=False (development), email is silently skipped and email_sent=false is returned

10.4 Hardware Interface Requirements

- Guard scanner page requires a physical camera device (USB webcam or built-in laptop camera)
- Browser must grant camera permission to the ScannerPage for html5-qrcode to function
- No other hardware dependencies — server runs on standard commodity hardware

11. System Constraints & Known Limitations

ID	Limitation	Impact	Workaround / Note
LIM-01	SQLite does not support concurrent writes	Not suitable for high-traffic production	Use PostgreSQL via DATABASE_URL in production
LIM-02	Email is sent synchronously within the API request	Slow SMTP delays response; no retry on failure	Set SMTP_ENABLED=False if unreliable; future: async queue
LIM-03	No visitor self-registration portal	Visitors cannot register themselves	Guards/admins must register all visitors
LIM-04	No real-time dashboard updates (no WebSocket)	Dashboard requires manual refresh	Planned: SSE or WebSocket in future version
LIM-05	QR images stored as base64 TEXT in database	Larger row size in visits table	Acceptable for office-scale traffic
LIM-06	No SMS or WhatsApp QR delivery	Visitors without email cannot receive digital QR	Guard can show QR on screen manually
LIM-07	No mobile native app	Guards must use browser on phone/tablet	Responsive web design mitigates this
LIM-08	No visitor self-check-in kiosk mode	Requires guard to operate scanner	Future: kiosk page with QR reader

12. Future Enhancements

The following features are outside the current v2.0 scope but are identified as high-value additions for future releases:

Priority	Enhancement	Description
High	Visitor Self-Registration	Public /register URL — visitor fills own form; host approves via email link before QR is issued
High	Async Email Queue	Celery + Redis worker sends emails async — removes SMTP latency from API response time
High	Real-Time Dashboard	WebSocket or SSE push for live check-in/out notifications without page refresh
Medium	OTP Fallback Check-In	Visitor receives OTP by phone; guard enters OTP if QR is not available
Medium	WhatsApp / SMS QR Delivery	Twilio or MSG91 integration to send QR as image to visitor's phone number
Medium	Docker Containerization	Dockerfile + docker-compose for one-command deployment across any environment
Medium	Alembic DB Migrations	Schema version control — safely migrate production DB without data loss
Low	Report Export (PDF/CSV)	Download visit reports as PDF or CSV from the admin dashboard
Low	Visitor Blacklist	Block specific visitors from checking in; alert guard on scan
Low	Mobile React Native App	Dedicated app for guards with offline QR scan capability

This SRS document represents the complete requirements baseline for VMS v2.0 as submitted for internship evaluation. All functional requirements listed in Section 5 are fully implemented and verifiable in the codebase.